

Strings and Character Array Manipulation in C

[DEF] Definition to Remember

In C, a string is a 1D array of characters terminated by a special null character ('\0' with ASCII value 0), manipulated via <string.h> library functions.

1. Concept Overview & Fundamentals

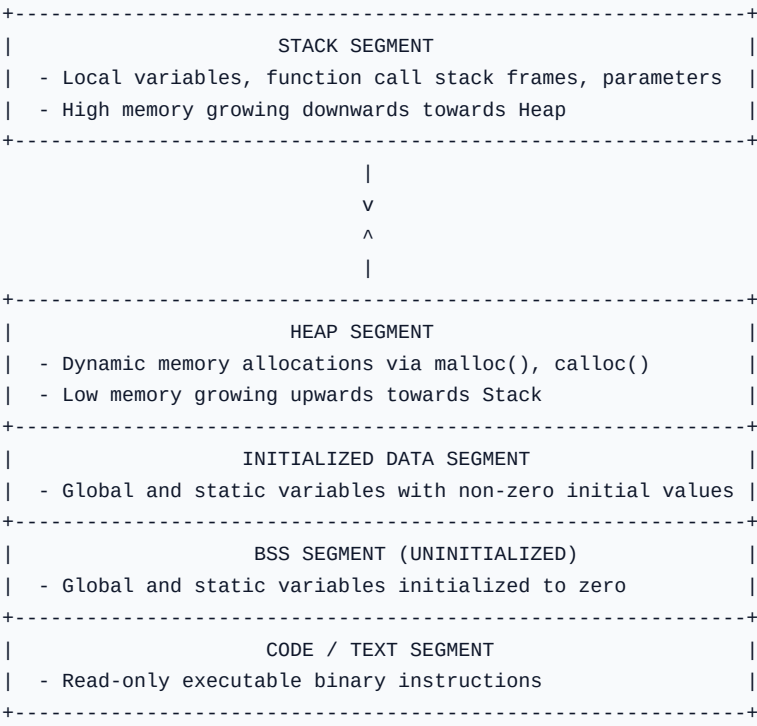
Strings and Character Array Manipulation in C forms a core cornerstone of computer systems programming, hardware-level software engineering, and competitive problem solving.

In the C execution model:

* Co

2. Technical Architecture & Memory Mechanics

Memory Layout & Storage Organization



Core Architecture Breakdown

Component / Concept	Technical Specification	Operational Characteristics	Performance / Complexity
Storage Allocation	Stack vs Heap vs Data Segment	Stack is ultra-fast LIFO; Heap offers	Stack: O(1) push/pop; Heap: O(1)
Type Checking	Static Typing	Compiler verifies types during compila	Zero runtime type penalty
Addressing Model	Byte-addressable physical RAM	Pointer stores hexadecimal byte memor	Direct memory dereference (O(1))
Execution Flow	Procedural / Sequential	Controlled via jumps, branches, loops,	Deterministic instruction cycles

3. Core Syntax & Implementation Patterns

Essential Idioms & Header Declarations

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

// Core standard implementation pattern
int main(void) {
    // Explicit initialization
    int status = 0;

    // Core logic execution
    printf("Executing C Module: Strings\n");

    return status;
}
```

4. Real-World Applications & System Engineering

* Operating System Kernels: Linux, UNIX, and Windows NT kernels utilize C for drivers, process scheduling, and memory paging.

* Em

5. Comprehensive Solved Problems & Code Walkthroughs

Problem 1: break keyword loop

Problem Description & Objective:

C Code Implementation:

Implem

```
/*
Create
from t
if a s
*/

#include
#include

int ma
/*
*
*
*
*
*
*
*
ch
wh
```

```
}  
pr  
re  
}
```

Problem 2: date format

Problem Description & Objective:

C Code Implementation:

Implem

```
/*  
Use pr  
and pr  
*/  
  
#inclu  
  
int ma  
/*  
*  
*  
*  
*  
*  
*  
*  
  
in  
in  
in  
  
pr  
  
re  
}
```

Problem 3: fgets puts

Problem Description & Objective:

C Code Implementation:

Implem

```
/*  
Read a  
and th  
*/  
  
#inclu
```

```
int ma
/*
*
*
*
*
*
*
*
ch
pr
fg
pr
pu
re
}
```

Problem 4: login system

Problem Description & Objective:

C Code Implementation:

Implem

```
/*
Create
that c
*/

#inclu
#inclu

int ma
/*
*
*
*
*
*
*
*
ch
ch
pr
sc
if
}
}
re
}
```

Problem 5: password checker

Problem Description & Objective:

C Code Implementation:

Implement

```

/*
Create
passwo
*/

#include
#include

int main()
/*
*
*
*
*
*
*
ch
do

}

pr
re
}

```

Problem 6: reverse string function

Problem Description & Objective:

C Code Implementation:

Implement

```

/*
Write
and re
*/

#include
#include

// Fun
void r
    /*
    *
    *
    *
    *
    *
    *
    */

```

```

*
st

in
in
ch

wh

}
}
int ma
ch

pr
fg

re

pr

re
}
```

Problem 7: string palindrome

Problem Description & Objective:

C Code Implementation:

Implem

```

/*
Create
string
*/

#include
#include

int ma
/*
*
*
*
*
*
*
ch
in
in

pr
fg

//
```

```
st
en
wh
}
if
}
}
re
}
```

Problem 8: tic tac toe board

Problem Description & Objective:

C Code Implementation:

Implem

```
/*
Use a
displa
*/

#inclu

int ma
/*
*
*
*
*
*
*
*

ch

};

in

pr

fo
```

```
}  
re  
}
```

Problem 9: trim string

Problem Description & Objective:

C Code Implementation:

Implem

```
/*  
Implem  
and tr  
*/  
  
#inclu  
#inclu  
#inclu  
  
// Fun  
void t  
/*  
*  
*  
*  
*  
*  
*  
*  
*  
*  
st  
  
in  
in  
in  
  
//  
wh  
  
}  
  
//  
wh  
  
}  
  
//  
fo  
  
}
```

```

    st
}

int ma
ch

pr
fg

tr

pr

re
}

```

Problem 10: uppercase string

Problem Description & Objective:

C Code Implementation:

Implem

```

/*
Write
to upp
*/

#include
#include

int ma
/*
*
*
*
*
*
*
*

ch
in

pr
fg

wh

}

pr

re
}

```

6. Exam Scoring Points & Memory Revision

[MUST-WRITE] Must-Write Points

1. Null-terminator ('\0'): Marks the end of string buffer; array size must be at least length + 1.
2. Standard functions in <string.h>: strlen(), strcpy(), strcat(), strcmp(), strrev().
3. Reading strings with spaces: Use fgets(str, size, stdin) instead of unsafe gets().
4. String comparison: strcmp(s1, s2) returns 0 if identical, <0 if s1 < s2, >0 if s1 > s2.
5. Character classification functions in <ctype.h>: isalpha(), isdigit(), toupper(), tolower().

[RECALL] Quick Recall

char array[] -> '\0' Null Terminator -> fgets(stdin) -> <string.h> Ops -> Bounds Safety